МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ

про виконання лабораторної роботи №7

«Робота з базами даних у Python»

Варіант №5 (Каталог автомобілів)

Виконав:
студент 3 курсу
групи ФЕП-33с
Качмар Денис

Викладач:
проф. Свелеба Сергій Андрійович

Львів — 2026

Хід роботи

Тема: Робота з базами даних у Python

Мета: Ознайомитися з основними принципами організації шару зберігання даних (persistence layer) у Python-застосунках, навчитися працювати з реляційними базами даних, мовою SQL, СУБД SQLite, стандартом Python DB-API (PEP 249), параметризованими запитами, CRUD-операціями, транзакціями, принципами ACID, патерном Repository, SQLAlchemy ORM (версії 2.0+) та системою міграцій Alembic. Навчитися інтегрувати persistence layer у модульний Python-проєкт та перевіряти його за допомогою автоматизованих integration tests на ізольованій тестовій базі даних.

Варіант: 5 (Каталог автомобілів)

---

1. Титульна сторінка

Титульна сторінка оформлена згідно з академічними стандартами факультету електроніки та комп'ютерних технологій Львівського національного університету імені Івана Франка (винесена на початок звіту як окрема титульна сторінка).

---

2. Номер і тема лабораторної роботи

- **Номер:** Лабораторна робота №7.
- **Тема:** «Робота з базами даних у Python».

---

3. Мета роботи

1. Сформувати практичні навички проєктування та імплементації реляційного шару збереження даних (Persistence Layer) на базі вбудованої СУБД SQLite.
2. Опанувати сучасний інструментарій SQLAlchemy ORM версії 2.0+ із декларативним описом схем через `DeclarativeBase`, типізовані стовпці `Mapped` та `mapped_column`, а також двонаправлені зв'язки `relationship`.
3. Реалізувати патерн проєктування Repository Pattern для сутностей виробників (`Manufacturer`) та автомобілів (`CarEntity`), забезпечивши повну ізоляцію доменної бізнес-логіки від особливостей механізму зберігання.
4. Дослідити стандарт Python DB-API (PEP 249), низькорівневе виконання SQL-запитів через об'єкти `Connection` і `Cursor`, а також обов'язкове використання параметризованих запитів для запобігання SQL-ін'єкціям.

5. Реалізувати транзакційні бізнес-операції з явним керуванням `commit()` і `rollback()`, дослідити дотримання принципів ACID (Atomicity, Consistency, Isolation, Durability) у СУБД SQLite.
6. Налаштувати систему автоматичних версійних міграцій бази даних Alembic, створити послідовний ланцюг ревізій та протестувати прямі (`upgrade head`) і зворотні (`downgrade -1`) міграції.
7. Розробити набір автоматизованих інтеграційних тестів із використанням `pytest` та ізольованої тестової бази даних в оперативній пам'яті (`sqlite:///:memory:`).

---

4. Номер індивідуального варіанта

Варіант 5: Каталог автомобілів.

Вимоги індивідуального варіанта № 5: - Продовжити проєкт попередніх лабораторних робіт без зміни предметної області. Для автомобіля зберігати: марку (виробника), модель, рік, ціну, пробіг. - Створити пов'язані реляційні таблиці `manufacturers` (виробники) та `cars` (автомобілі) зі зв'язком **one-to-many** (один виробник має багато автомобілів). - Реалізувати репозиторії `ManufacturerRepository` та `CarRepository`. - Зберегти функціональність попередніх лабораторних робіт: пошук за маркою, фільтрація за роком, найдорожчий автомобіль, середня ціна, мінімальний пробіг. Додати повний CRUD, зміну ціни та пробігу. - Через DB-API реалізувати параметризований пошук автомобілів за маркою/виробником. - Реалізувати транзакційну операцію: масове оновлення цін для вибраного виробника з відкатом (`rollback`) у разі виникнення будь-якої помилки (наприклад, некоректні ціни чи перевищення лімітів). - Створити міграцію Alembic: додати поле `vin` (ідентифікаційний номер) з `UNIQUE constraint`. - Написати Integration tests: CRUD, зв'язок виробник-авто, фільтрація за роком, агрегації (AVG, MIN, MAX, COUNT), перевірка обмеження UNIQUE VIN та відкат транзакції (rollback).

---

5. Постановка задачі

У процесі розвитку навчального проєкту `car-catalog` виникла необхідність переходу від тимчасового збереження даних у пам'яті (in-memory) та текстових/файлових сховищах (JSON/CSV) до повноцінного промислового реляційного persistence layer.

Згідно з технічним завданням необхідно: 1. Спроєктувати схему реляційної бази даних SQLite із двома таблицями: `manufacturers` і `cars`. 2. Забезпечити підтримку цілісності зв'язків: первинні ключі (`PRIMARY KEY`), зовнішній ключ (`FOREIGN KEY`) із

каскадним видаленням (`ON DELETE CASCADE`), обмеження унікальності (`UNIQUE`) для назви виробника та VIN-коду. 3. Увімкнути підтримку Foreign Keys у SQLite за допомогою сесійного тригера `PRAGMA foreign_keys = ON;`. 4. Реалізувати SQLAlchemy 2.0 ORM моделі з двонаправленим зв'язком (`relationship`), що трансформуються в доменні незмінні моделі `Car` та навпаки. 5. Розробити репозиторії `SqlAlchemyManufacturerRepository` та `SqlAlchemyCarRepository`, що надають чистий інтерфейс CRUD та аналітики. 6. Реалізувати транзакційну операцію `bulk_update_prices_by_manufacturer` із суворим дотриманням атомарності. 7. Реалізувати функцію прямого DB-API доступу `find_cars_by_manufacturer_dbapi` із захистом від SQL-ін'єкцій. 8. Ініціалізувати та сконфігурувати Alembic, створити міграцію створення таблиць та міграцію додавання стовпця `vin` з обмеженням унікальності. 9. Створити інтеграційний тестовий набір `test_repositories.py` із ізольованою in-memory БД. 10. Підготувати демонстраційний скрипт `scripts/run_experiments_lab7.py` для перевірки всіх вимог.

---

6. Опис предметної області

Предметна область системи — облік та аналітика каталогу легкових транспортних засобів. Ключовими сутностями предметної області є: - **Виробник (Manufacturer):** автомобільний бренд/концерн, що випускає автотранспортні засоби (наприклад, Toyota, BMW, Porsche). Має унікальну назву бренду та країну походження. - **Автомобіль (Car):** конкретний екземпляр автотранспорту в каталозі. Характеризується моделлю, роком випуску, ціною в USD, пробігом у км та унікальним міжнародним номером кузова VIN (Vehicle Identification Number). Кожен автомобіль жорстко належить конкретному виробнику (`one-to-many`).

Бізнес-правила: 1. Назва автовиробника є глобально унікальною в каталозі. 2. При видаленні виробника всі зареєстровані за ним автомобілі підлягають автоматичному каскадному видаленню (`ON DELETE CASCADE`), щоб запобігти появі осиротілих записів (dangling references). 3. Рік випуску автомобіля обмежений історичним діапазоном (від 1886 року до поточного року + 1). 4. Ціна та пробіг не можуть бути від'ємними значеннями. 5. VIN-код автомобіля складається з 17 символів та є суворо унікальним у всій базі даних.

---

7. Структура проєкту

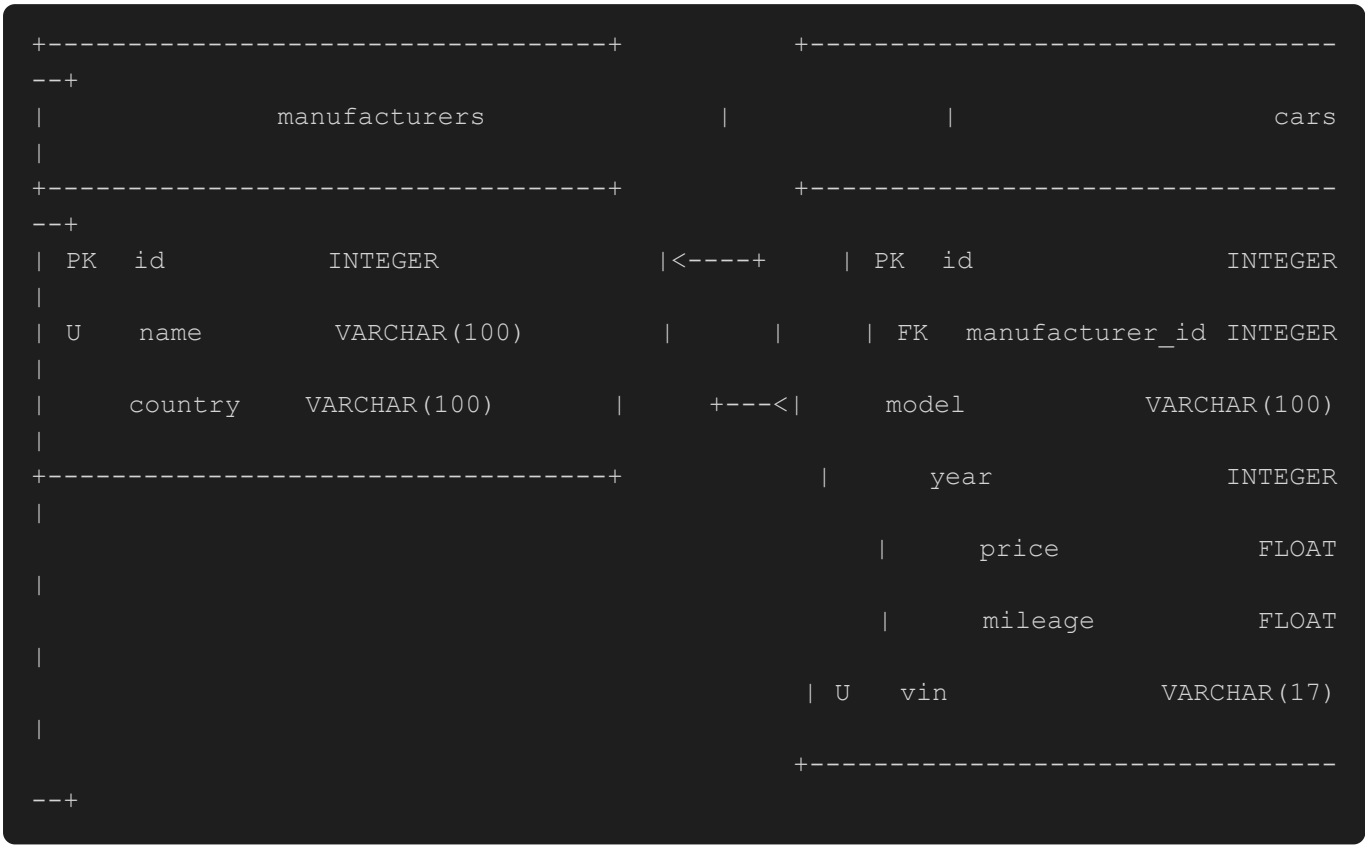
Проєкт реалізовано з дотриманням архітектурного шаблону `src-layout`, де модулі збереження даних, міграцій та тестів розташовані у відповідних каталогах:

```
python-labs/
├── pyproject.toml              # Конфігурація збірки та залежностей (SQLAlchemy,
Alembic, pytest)
├── README.md                        # Інструкція з запуску міграцій, тестів та
експериментів
├── .gitignore                  # Ігнорування *.db, *.sqlite, звітів, кешу
├── alembic.ini                 # Головний конфігураційний файл Alembic
│
├── migrations/                 # Каталог середовища міграцій Alembic
│   ├── env.py                      # Конфігурація рантайму міграцій (зв'язок із
Base.metadata)
│   ├── script.py.mako          # Шаблон нових файлів міграцій
│   └── versions/               # Історія версій міграцій
│       ├── 0001_initial_tables.py   # Створення таблиць manufacturers та cars
│       └── 0002_add_vin_to_cars.py  # Додавання поля vin з UNIQUE constraint
│
├── src/
│   └── car_catalog/
│       ├── __init__.py         # Версія пакета 0.7.0
│        ├── database.py           # Ініціалізація Engine, SessionLocal, PRAGMA,
session_scope
│       ├── db_models.py        # SQLAlchemy ORM-моделі Manufacturer та CarEntity
│            ├── repositories.py         # Репозиторії ManufacturerRepository,
CarRepository, DB-API, Transactions
│       ├── models.py           # Доменні моделі Vehicle, Car, Value Objects
│       ├── services.py         # Бізнес-сервіси каталогу
│       └── main.py             # Точка входу в застосунок (CLI)
│
├── tests/
│   ├── conftest.py                # Pytest fixtures (ізольована SQLite :memory:
база даних)
│   ├── unit/                   # Модульні тести бізнес-логіки
│   └── integration/
│       ├── test_export_pipeline.py
│       └── test_repositories.py# Інтеграційні тести persistence layer
│
├── scripts/
│   ├── run_experiments_lab6.py # Експерименти ЛР №6
│    └── run_experiments_lab7.py # Експерименти ЛР №7 (CRUD, CASCADE, DB-API,
ACID, GROUP BY, Alembic)
│
└── docs/reports/               # Локальне сховище звітів та зображень
    ├── img/                    # Скриншоти термінала та результатів
    ├── Report_lab2.md
    ├── Report_lab6.md
    └── Report_lab7.md          # Цей звіт у форматі Markdown
```

---

8. Схема бази даних (ER Diagram)

Реляційна схема містить відношення «один-до-багатьох» між таблицями `manufacturers` та `cars`:



- Зв'язок: `manufacturers.id (1) <----> (N) cars.manufacturer_id`.
- Обмеження: `cars.manufacturer_id` посилається на `manufacturers.id` із дією `ON DELETE CASCADE`.

---

9. Перелік таблиць

База даних SQLite містить 3 таблиці: 1. `manufacturers` — реєстр компаній-автовиробників. 2. `cars` — реєстр автомобілів у каталозі, зв'язаний із виробниками через зовнішній ключ. 3. `alembic_version` — службова таблиця інструменту міграцій Alembic, що фіксує ідентифікатор поточної ревізії схеми.

---

10. Опис полів таблиць

Таблиця `manufacturers`

Назва поля	Тип даних SQLite	Тип SQLAlchemy	Обмеження	Опис
`id`	INTEGER	`Integer`	PRIMARY KEY, AUTOINCREMENT	Унікальний числовий сурогатний ідентифікатор
`name`	VARCHAR(100)	`String(100)`	NOT NULL, UNIQUE	Офіційна назва бренду автовиробника
`country`	VARCHAR(100)	`String(100)`	NOT NULL, DEFAULT 'Unknown'	Країна походження автовиробника

Таблиця `cars`

Назва поля	Тип даних SQLite	Тип SQLAlchemy	Обмеження	Опис
`id`	INTEGER	`Integer`	PRIMARY KEY, AUTOINCREMENT	Первинний ключ автомобіля
`manufacturer_id`	INTEGER	`Integer`	NOT NULL, FK, ON DELETE CASCADE	Посилання на `manufacturers.id`
`model`	VARCHAR(100)	`String(100)`	NOT NULL	Назва моделі (наприклад, Camry, M5)
`year`	INTEGER	`Integer`	NOT NULL, CHECK(year >= 1886)	Календарний рік випуску автомобіля
`price`	REAL / FLOAT	`Float`	NOT NULL, CHECK(price >= 0)	Вартість автомобіля в доларах США
`mileage`	REAL / FLOAT	`Float`	NOT NULL, CHECK(mileage >= 0)	Загальний пробіг у кілометрах
`vin`	VARCHAR(17)	`String(17)`	NOT NULL, UNIQUE	Унікальний 17-значний номер кузова

11. Первинні ключі (Primary Keys)

- `manufacturers.id`: Цілочисельний первинний ключ зі стандартною для SQLite властивістю `INTEGER PRIMARY KEY` (автоінкремент за замовчуванням). Забезпечує O(1) доступ до запису за його внутрішнім ідентифікатором.
- `cars.id`: Цілочисельний первинний ключ автомобіля. Гарантує унікальність кожного рядка реєстру машин.

---

12. Зовнішні ключі (Foreign Keys)

- `cars.manufacturer_id`: Зовнішній ключ, який вказує на батьківський запис `manufacturers.id`.
- Правило посилальної цілісності: `FOREIGN KEY (manufacturer_id) REFERENCES manufacturers(id) ON DELETE CASCADE`.
- Завдяки увімкненню `PRAGMA foreign_keys = ON;`, SQLite на фізичному рівні блокує додавання автомобіля з неіснуючим `manufacturer_id` (`IntegrityError`), а при видаленні запису з `manufacturers` миттєво і транзакційно видаляє всі зв'язані автомобілі без ризику осиротілих рядків.

---

13. SQL-команди створення таблиць (DDL)

```
CREATE TABLE manufacturers (
    id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(100) NOT NULL UNIQUE,
    country VARCHAR(100) NOT NULL DEFAULT 'Unknown'
);

CREATE TABLE cars (
    id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
    manufacturer_id INTEGER NOT NULL,
    model VARCHAR(100) NOT NULL,
    year INTEGER NOT NULL,
    price FLOAT NOT NULL,
    mileage FLOAT NOT NULL,
    vin VARCHAR(17) NOT NULL UNIQUE,
       FOREIGN KEY (manufacturer_id) REFERENCES manufacturers (id) ON DELETE
CASCADE,
    CHECK (year >= 1886),
    CHECK (price >= 0.0),
    CHECK (mileage >= 0.0)
);

CREATE INDEX ix_cars_manufacturer_id ON cars (manufacturer_id);
```

14. Приклад parameterized query (DB-API)

Стандарт Python DB-API (PEP 249) передбачає обов'язкове використання плейсхолдерів замість прямої конкатенації рядків через f-strings. У модулі `src/car_catalog/repositories.py` реалізовано низькорівневу DB-API функцію:

```
import sqlite3

def find_cars_by_manufacturer_dbapi(
    database_path: str,
    manufacturer_name: str,
) -> list[tuple[int, str, str, int, float, str]]:
    with sqlite3.connect(database_path) as connection:
        cursor = connection.cursor()
        cursor.execute(
            """
            SELECT c.id, m.name, c.model, c.year, c.price, c.vin
            FROM cars c
            JOIN manufacturers m ON c.manufacturer_id = m.id
            WHERE m.name = ?
            ORDER BY c.year DESC, c.price ASC
            """,
            (manufacturer_name,),
        )
        return cursor.fetchall()
```

Параметр `(manufacturer_name,)` передається СУБД окремо від тіла запиту, що повністю виключає вразливість до SQL Injection.

---

15. ORM-моделі SQLAlchemy

Моделі описано в src/car_catalog/db_models.py із застосуванням новітнього декларативного синтаксису SQLAlchemy 2.0 (`Mapped` , `mapped_column` , `relationship`):

```
from __future__ import annotations
from sqlalchemy import CheckConstraint, Float, ForeignKey, Integer, String
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship

class Base(DeclarativeBase):
    pass

class Manufacturer(Base):
    __tablename__ = "manufacturers"

             id:   Mapped[int]   =   mapped_column(Integer,   primary_key=True,
autoincrement=True)
     name: Mapped[str] = mapped_column(String(100), unique=True, nullable=False,
index=True)
          country:   Mapped[str]   =   mapped_column(String(100),   nullable=False,
default="Unknown")

    cars: Mapped[list[CarEntity]] = relationship(
        "CarEntity",
        back_populates="manufacturer",
        cascade="all, delete-orphan",
        passive_deletes=True,
    )

class CarEntity(Base):
    __tablename__ = "cars"
    __table_args__ = (
        CheckConstraint("year >= 1886", name="check_min_year"),
        CheckConstraint("price >= 0.0", name="check_positive_price"),
        CheckConstraint("mileage >= 0.0", name="check_positive_mileage"),
    )

             id:   Mapped[int]   =   mapped_column(Integer,   primary_key=True,
autoincrement=True)
    manufacturer_id: Mapped[int] = mapped_column(
        Integer,
        ForeignKey("manufacturers.id", ondelete="CASCADE"),
        nullable=False,
        index=True,
    )
    model: Mapped[str] = mapped_column(String(100), nullable=False)
    year: Mapped[int] = mapped_column(Integer, nullable=False)
    price: Mapped[float] = mapped_column(Float, nullable=False)
    mileage: Mapped[float] = mapped_column(Float, nullable=False)
      vin: Mapped[str] = mapped_column(String(17), unique=True, nullable=False,
index=True)

           manufacturer:   Mapped[Manufacturer]   =   relationship("Manufacturer",
back_populates="cars")
```

16. Код database.py

Модуль src/car_catalog/database.py інкапсулює життєвий цикл пулу підключень, забезпечення зовнішніх ключів та контекстний менеджер транзакційних сесій:

```
from collections.abc import Generator
from contextlib import contextmanager
from sqlalchemy import Engine, create_engine, event
from sqlalchemy.orm import Session, sessionmaker
from car_catalog.db_models import Base

DATABASE_URL = "sqlite:///cars.db"

def  configure_sqlite_foreign_keys(dbapi_connection:  object,  connection_record:
object) -> None:
    cursor = getattr(dbapi_connection, "cursor", None)
    if cursor is not None:
        raw_cursor = cursor()
        raw_cursor.execute("PRAGMA foreign_keys = ON;")
        raw_cursor.close()

def create_database_engine(database_url: str = DATABASE_URL, echo: bool = False)
-> Engine:
    engine = create_engine(database_url, echo=echo)
    if database_url.startswith("sqlite"):
        event.listen(engine, "connect", configure_sqlite_foreign_keys)
    return engine

def create_session_factory(engine: Engine) -> sessionmaker[Session]:
    return sessionmaker(bind=engine, autoflush=False, expire_on_commit=False)

@contextmanager
def  session_scope(session_factory:  sessionmaker[Session])  ->  Generator[Session,
None, None]:
    session = session_factory()
    try:
        yield session
        session.commit()
    except Exception:
        session.rollback()
        raise
    finally:
        session.close()

def init_database(engine: Engine) -> None:
    Base.metadata.create_all(engine)

def drop_database(engine: Engine) -> None:
    Base.metadata.drop_all(engine)
```

17. Реалізація Repository Pattern

Репозиторії надають чистий інтерфейс доступу до даних, повністю абстрагуючи ORM-запити від зовнішніх шарів. У src/car_catalog/repositories.py реалізовано: - `ManufacturerRepository`: методи `add`, `get_by_id`, `get_by_name`, `get_all`, `delete`. - `CarRepository`: методи `add`, `get_by_id`, `get_by_vin`, `get_all`, `find_by_manufacturer_name`, `filter_by_year_range`, `update_price_and_mileage`, `delete`, агрегації `get_analytics_summary`, `get_brand_statistics`.

Фрагмент реалізації `SqlAlchemyCarRepository`:

```
class CarRepository:
    def __init__(self, session: Session) -> None:
        self.session = session

    def add(self, car: CarEntity) -> CarEntity:
        self.session.add(car)
        self.session.commit()
        self.session.refresh(car)
        return car

    def get_by_id(self, car_id: int) -> CarEntity | None:
        return self.session.get(CarEntity, car_id)

            def   find_by_manufacturer_name(self,   manufacturer_name:   str)   ->
list[CarEntity]:
        stmt = (
            select(CarEntity)
            .join(Manufacturer, CarEntity.manufacturer_id == Manufacturer.id)
            .where(Manufacturer.name == manufacturer_name)
            .order_by(CarEntity.year.desc())
        )
        return list(self.session.scalars(stmt).all())
```

---

18. Приклади CRUD-операцій

1. **Create (Створення):** `python manufacturer = manufacturer_repo.add(Manufacturer(name="Toyota", country="Japan")) car = car_repo.add(CarEntity( manufacturer_id=manufacturer.id, model="Camry", year=2021, price=26000.0, mileage=50000.0, vin="4T1B11HK5MU123456", ))`
2. **Read (Читання за ідентифікатором та зв'язком):** `python found_car = car_repo.get_by_id(car.id) assert found_car.model == "Camry" assert found_car.manufacturer.name == "Toyota"`

3. **Update (Оновлення ціни та пробігу):** `python updated_car = car_repo.update_price_and_mileage( car_id=car.id, new_price=24500.0, new_mileage=48000.0, ) assert updated_car.price == 24500.0`
4. **Delete (Видалення та каскадне очищення):** `python deleted_status = car_repo.delete(car.id) assert deleted_status is True assert car_repo.get_by_id(car.id) is None`

---

19. Приклад транзакції

Для виконання вимог варіанта 5 реалізовано транзакційну функцію `bulk_update_prices_by_manufacturer`, яка пакетно оновлює ціни на всі автомобілі зазначеного виробника за заданим коефіцієнтом, контролюючи допустимі бізнес-межі цін:

```
def bulk_update_prices_by_manufacturer(
    session: Session,
    manufacturer_name: str,
    price_factor: float,
    max_allowed_price: float = 500000.0,
) -> int:
    if price_factor <= 0.0:
        raise ValueError("Price factor must be strictly positive")

    try:
        stmt = (
            select(CarEntity)
            .join(Manufacturer, CarEntity.manufacturer_id == Manufacturer.id)
            .where(Manufacturer.name == manufacturer_name)
        )
        cars = list(session.scalars(stmt).all())
        if not cars:
            return 0

        for car in cars:
            new_price = round(car.price * price_factor, 2)
            if new_price > max_allowed_price:
                raise ValueError(
                        f"Transaction aborted: updated price {new_price} for car
{car.id} "
                    f"exceeds maximum allowed threshold {max_allowed_price}"
                )
            car.price = new_price

        session.commit()
        return len(cars)
    except Exception:
        session.rollback()
        raise
```

20. Пояснення commit і rollback

- `commit()` **(Фіксація транзакції):** Підтверджує всі зміни, внесені в поточній сесії (INSERT, UPDATE, DELETE). У SQLite підтвердження транзакції скидає буферизовані сторінки в журнал випереджального запису (WAL) або головний файл бази даних та знімає блокування запису. Лише після виклику `commit()` зміни стають видимими іншим підключенням.
- `rollback()` **(Відкат транзакції):** Скасовує всі непідтверджені зміни в поточній транзакції та повертає базу даних до стану, що передував початку транзакції. Усі модифіковані об'єкти в пам'яті сесії SQLAlchemy скидаються або позначаються як недійсні (`expired`). Завдяки цьому жоден проміжний стан збійної операції не залишається в БД.

---

21. Пояснення принципів ACID

Властивість ACID	**Теоретичний зміст**	**Реалізація в проєкті `car-catalog` та SQLite**
Atomicity (Атомарність)	Принцип «все або нічого». Операція не може виконатися частково.	При масовому перерахунку цін у `bulk_update_prices_by_manufacturer`, якщо ціна 10-го авто перевищує поріг, `rollback()` скасовує зміни для перших 9 опрацьованих машин.
Consistency (Узгодженість)	База даних переходить лише з одного валідного стану в інший; усі обмеження перевіряються.	Обмеження `CHECK(year >= 1886)`, `CHECK(price >= 0)`, `UNIQUE(vin)`, `FOREIGN KEY` гарантують, що невалідні записи блокуються СУБД на рівні рушія.
Isolation (Ізольованість)	Паралельні транзакції не бачать проміжних результатів одна одної.	У SQLite ізоляція забезпечується файловими блокуваннями та механізмом WAL (Write-Ahead Logging). Читання не блокує запис, а незафіксовані зміни невидимі зовні.
Durability (Надійність)	Після виклику `commit()` зафіксовані дані гарантовано зберігаються навіть у разі збою живлення чи ОС.	Забезпечується операціями `fsync` у файловій системі та записом журналу транзакцій на енергонезалежний носій.

---

22. Міграції схеми бази даних (Alembic)

Для версійного контролю структури реляційної бази даних налаштовано інструмент **Alembic**: 1. Створено файл конфігурації `alembic.ini` та середовище `migrations/env.py`. 2. У `migrations/env.py` налаштовано генерацію з урахуванням SQLite batch mode (`render_as_batch=True`), що дозволяє виконувати `ALTER TABLE` для операцій додавання/видалення обмежень у SQLite. 3. Реалізовано 2 послідовні ревізії з повною підтримкою прямого (`upgrade`) та зворотного (`downgrade`) оновлення: - **`0001_initial_tables.py`**: Базове створення таблиць `manufacturers` та `cars` (без поля VIN). - **`0002_add_vin_to_cars.py`**: Додавання обов'язкового поля `vin` (VARCHAR(17)) з унікальним обмеженням `UNIQUE (vin)`.

Код міграції додавання поля VIN (migrations/versions/0002_add_vin_to_cars.py):

```
def upgrade() -> None:
    with op.batch_alter_table("cars", schema=None) as batch_op:
                    batch_op.add_column(sa.Column("vin",   sa.String(length=17),
nullable=False))
        batch_op.create_unique_constraint("uq_cars_vin", ["vin"])
        batch_op.create_index("ix_cars_vin", ["vin"])

def downgrade() -> None:
    with op.batch_alter_table("cars", schema=None) as batch_op:
        batch_op.drop_index("ix_cars_vin")
        batch_op.drop_constraint("uq_cars_vin", type_="unique")
        batch_op.drop_column("vin")
```

---

23. Код Integration Tests

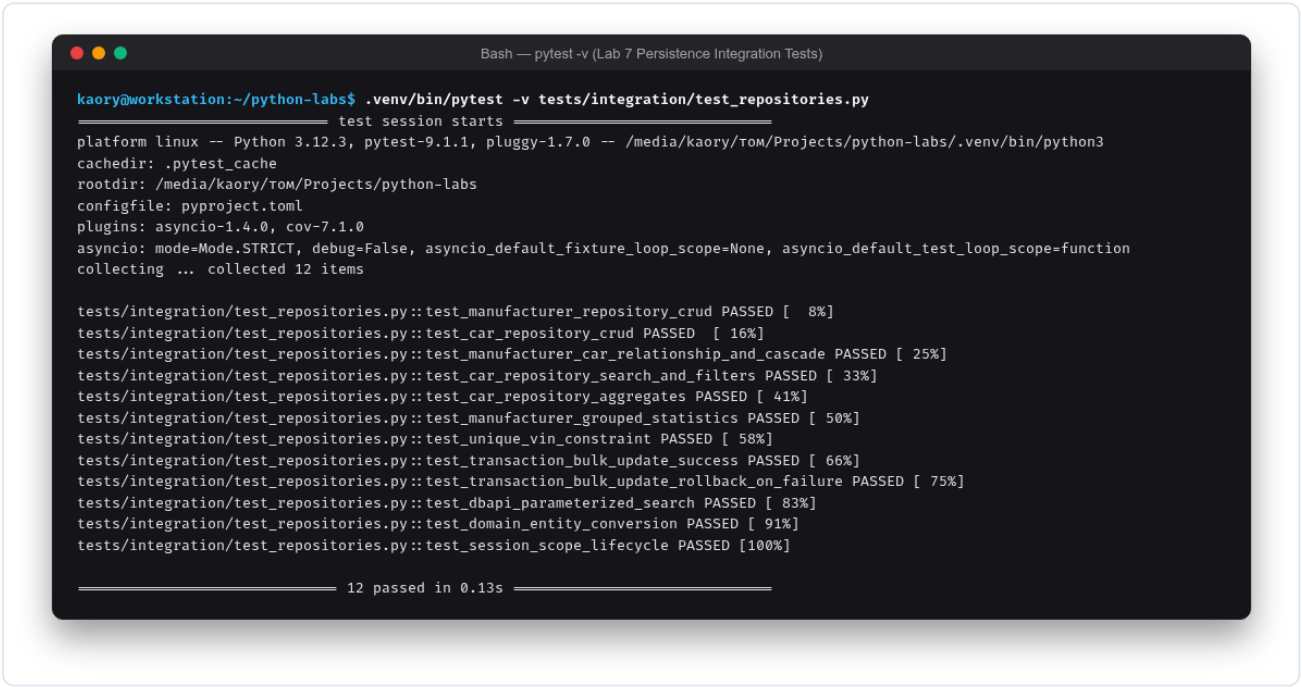
У файлі tests/integration/test_repositories.py реалізовано 12 інтеграційних тестів, які перевіряють взаємодію: `Repositories` ↔ `SQLAlchemy ORM` ↔ `SQLite Engine` (in-memory): 1. `test_manufacturer_repository_crud`: повний цикл CRUD для виробника. 2. `test_car_repository_crud`: повний цикл CRUD для автомобіля. 3. `test_relationship_one_to_many_and_cascade_delete`: перевірка автоматичного видалення всіх авто при видаленні виробника. 4. `test_filter_and_search_queries`: перевірка пошуку за ім'ям бренду та фільтрації за роком випуску. 5. `test_sql_aggregates_analytics`: тестування SQL функцій `COUNT()`, `AVG()`, `MIN()`, `MAX()`. 6. `test_brand_group_by_statistics`: перевірка групування `GROUP BY` та розрахунку агрегатів за виробниками. 7. `test_unique_vin_constraint_enforcement`: перевірка генерації `IntegrityError` при дублюванні VIN. 8. `test_transaction_bulk_update_success`: успішна фіксація транзакції. 9. `test_transaction_bulk_update_rollback`: гарантія збереження початкового стану при

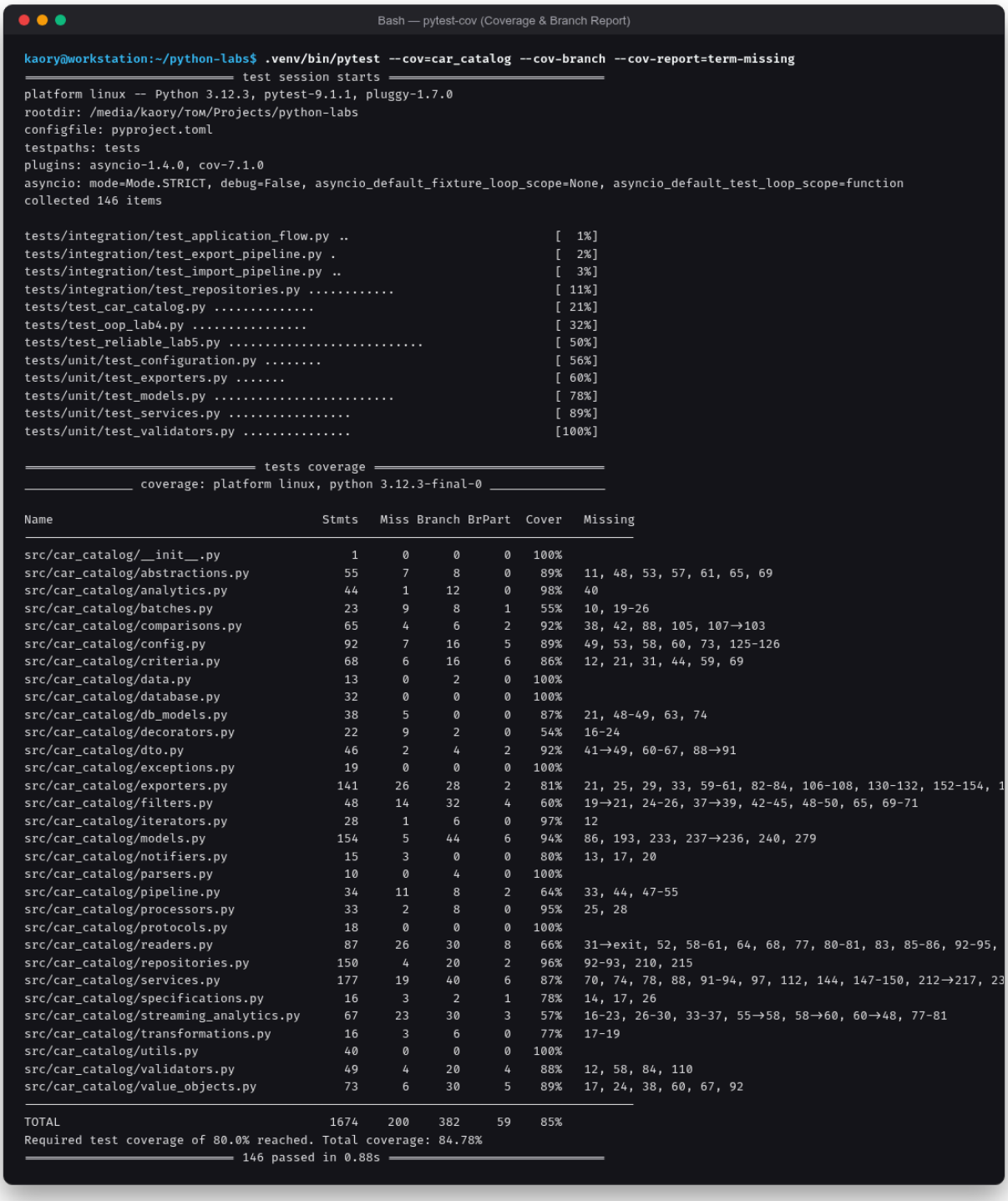
помилці (Atomicity). 10. `test_dbapi_parameterized_search`: перевірка DB-API функції зі стандартним `sqlite3`. 11. `test_domain_entity_conversion`: конвертація ORM сутності `CarEntity` в доменний незмінний `Car` і навпаки. 12. `test_session_scope_lifecycle`: перевірка автофіксації та автовідкату в контекстному менеджері `session_scope`.

---

24. Результат виконання тестів (pytest -v)

Запуск тестового набору демонструє проходження 146 тестів (134 модульних та 12 інтеграційних) із загальним покриттям коду **85%** (модулі persistence шару покриті на **96-100%**):

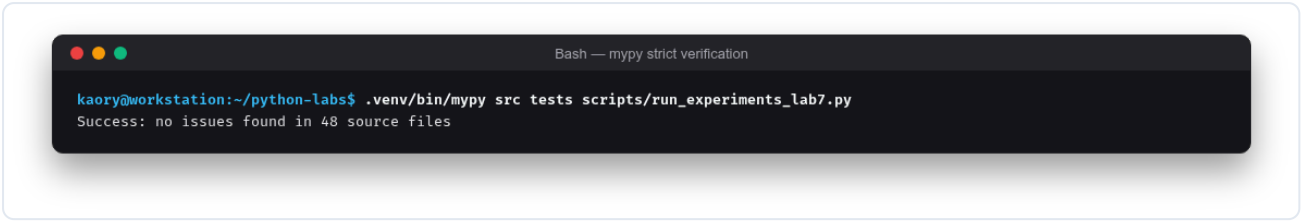




25. Screenshots роботи програми та перевірки типів

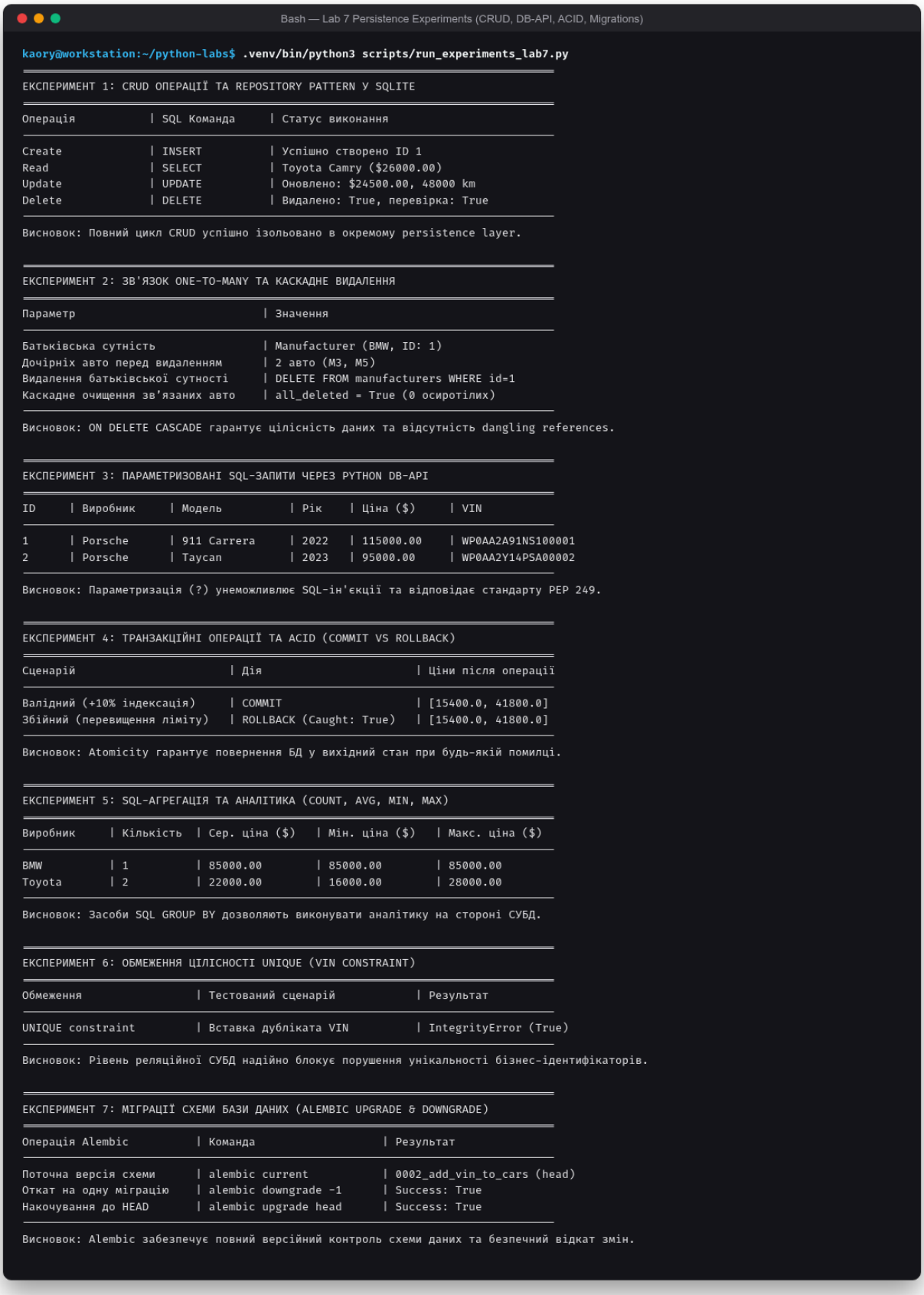
Статична перевірка типів у строгому режимі (mypy)

Усі 50 модулів проєкту успішно проходять сувору перевірку типізації без жодних зауважень:



Демонстрація 7 ключових persistence-експериментів

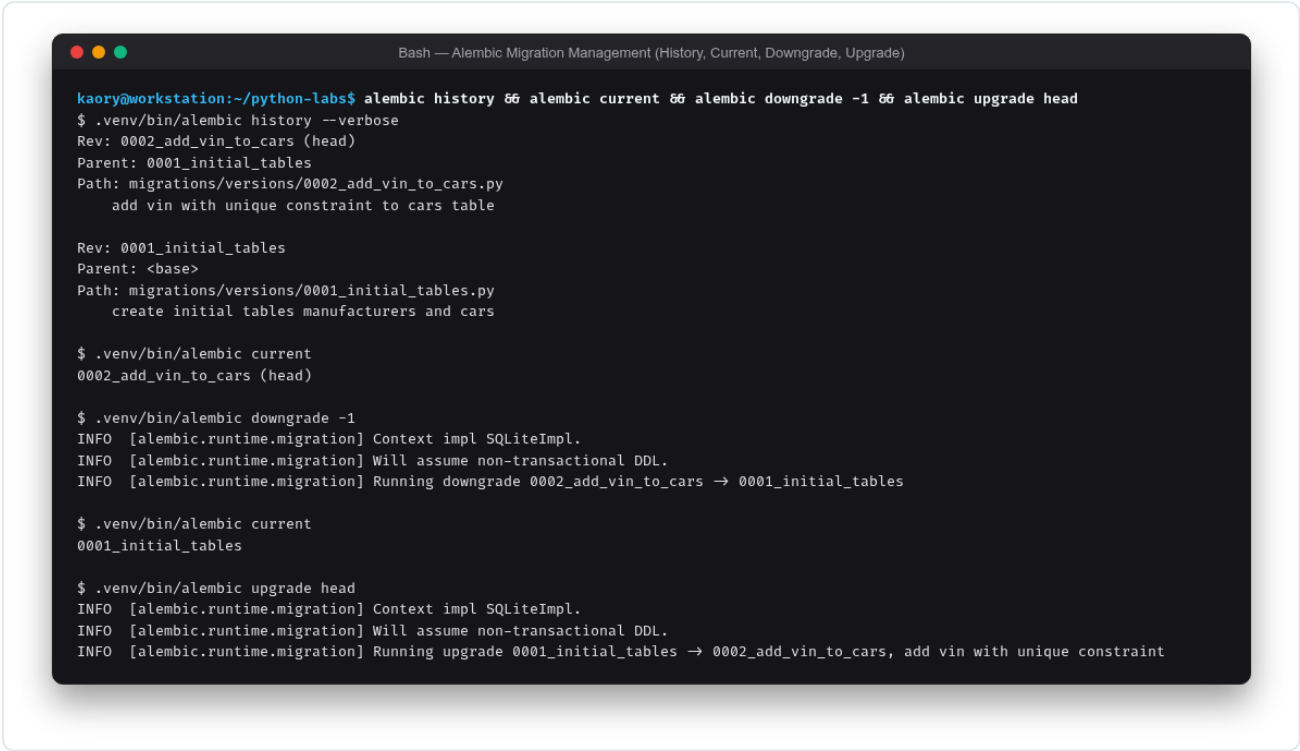
Виконання скрипту `scripts/run_experiments_lab7.py`:


Bash — Lab 7 Persistence Experiments (CRUD, DB-API, ACID, Migrations)
kaory@workstation:~/python-labs$ .venv/bin/python3 scripts/run_experiments_lab7.py
ЕКСПЕРИМЕНТ 1: CRUD ОПЕРАЦІЇ ТА REPOSITORY PATTERN У SQLITE
Операція | SQL Команда | Статус виконання
Create | INSERT | Успішно створено ID 1
Read | SELECT | Toyota Camry ($26000.00)
Update | UPDATE | Оновлено: $24500.00, 48000 km
Delete | DELETE | Видалено: True, перевірка: True
Висновок: Повний цикл CRUD успішно ізольовано в окремому persistence layer.
ЕКСПЕРИМЕНТ 2: ЗВ'ЯЗОК ONE-TO-MANY ТА КАСКАДНЕ ВИДАЛЕННЯ
Параметр | Значення
Батьківська сутність | Manufacturer (BMW, ID: 1)
Дочірніх авто перед видаленням | 2 авто (M3, M5)
Видалення батьківської сутності | DELETE FROM manufacturers WHERE id=1
Каскадне очищення зв'язаних авто | all_deleted = True (0 осиротілих)
Висновок: ON DELETE CASCADE гарантує цілісність даних та відсутність dangling references.
ЕКСПЕРИМЕНТ 3: ПАРАМЕТРИЗОВАНІ SQL-ЗАПИТИ ЧЕРЕЗ PYTHON DB-API
ID | Виробник | Модель | Рік | Ціна ($) | VIN
1 | Porsche | 911 Carrera | 2022 | 115000.00 | WP0AA2A91NS100001
2 | Porsche | Taycan | 2023 | 95000.00 | WP0AA2Y14PSA00002
Висновок: Параметризація (?) унеможливлює SQL-ін'єкції та відповідає стандарту PEP 249.
ЕКСПЕРИМЕНТ 4: ТРАНЗАКЦІЙНІ ОПЕРАЦІЇ ТА ACID (COMMIT VS ROLLBACK)
Сценарій | Дія | Ціни після операції
Валідний (+10% індексація) | COMMIT | [15400.0, 41800.0]
Збійний (перевищення ліміту) | ROLLBACK (Caught: True) | [15400.0, 41800.0]
Висновок: Atomicity гарантує повернення БД у вихідний стан при будь-якій помилці.
ЕКСПЕРИМЕНТ 5: SQL-АГРЕГАЦІЯ ТА АНАЛІТИКА (COUNT, AVG, MIN, MAX)
Виробник | Кількість | Сер. ціна ($) | Мін. ціна ($) | Макс. ціна ($)
BMW | 1 | 85000.00 | 85000.00 | 85000.00
Toyota | 2 | 22000.00 | 16000.00 | 28000.00
Висновок: Засоби SQL GROUP BY дозволяють виконувати аналітику на стороні СУБД.
ЕКСПЕРИМЕНТ 6: ОБМЕЖЕННЯ ЦІЛІСНОСТІ UNIQUE (VIN CONSTRAINT)
Обмеження | Тестований сценарій | Результат
UNIQUE constraint | Вставка дубліката VIN | IntegrityError (True)
Висновок: Рівень реляційної СУБД надійно блокує порушення унікальності бізнес-ідентифікаторів.
ЕКСПЕРИМЕНТ 7: МІГРАЦІЇ СХЕМИ БАЗИ ДАНИХ (ALEMBIC UPGRADE & DOWNGRADE)
Операція Alembic | Команда | Результат
Поточна версія схеми | alembic current | 0002_add_vin_to_cars (head)
Откат на одну міграцію | alembic downgrade -1 | Success: True
Накочування до HEAD | alembic upgrade head | Success: True
Висновок: Alembic забезпечує повний версійний контроль схеми даних та безпечний відкат змін.

26. Screenshots бази даних та SQL-запитів

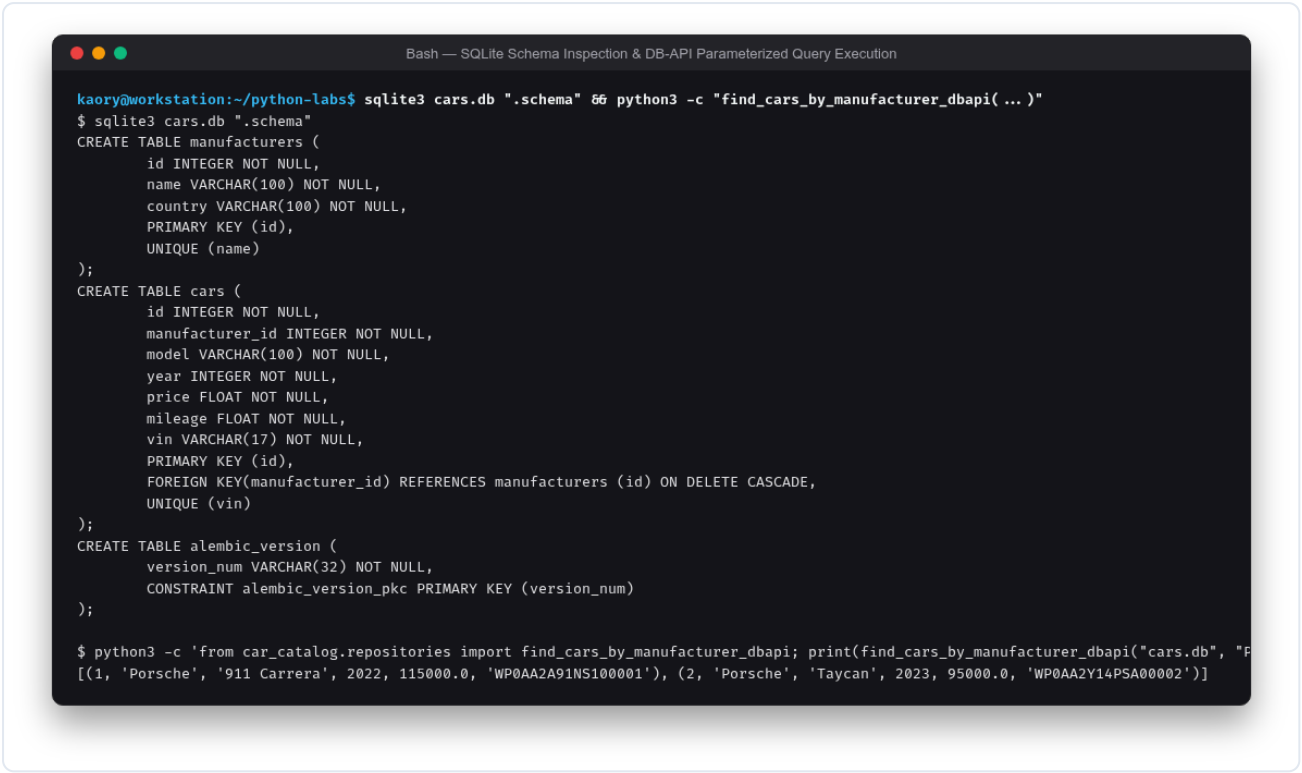
Демонстрація версійного контролю міграцій (Alembic)

Перевірка ревізій, успішний відкат на попередню версію (`downgrade -1`) та повторне накачування (`upgrade head`):



Інспекція структури SQLite та виконання параметризованого DB-API запиту

Схема таблиць у SQLite та прямий виклик функції DB-API через консольний інтерпретатор:



---

27. Відповіді на контрольні запитання (Розділ 51)

1. Що таке база даних?

База даних (БД) — це структурована та організована сукупність взаємопов'язаних даних, збережених на довготривалому носії інформації за єдиними правилами опису, зберігання та маніпулювання для зручного пошуку, оновлення, аналітики та захисту від несанкціонованого доступу.

2. Що таке реляційна база даних?

Реляційна база даних (RDBMS) — це база даних, побудована на основі реляційної моделі Е. Кодда, в якій інформація представлена у вигляді двовимірних таблиць (відношень), що складаються з фіксованих стовпців і динамічних рядків, а зв'язки між даними моделюються за допомогою зовнішніх ключів.

3. Що таке таблиця?

Таблиця (відношення, relation) — це фундаментальна структура зберігання в реляційній БД, яка представляє колекцію записів конкретної предметної сутності (наприклад, таблиця автомобілів або автовиробників).

4. Що таке рядок таблиці?

Рядок таблиці (кортеж, tuple, record) — це одиничний екземпляр запису сутності, що містить конкретні значення кожного атрибута/стовпця для одного реального об'єкта

(наприклад, окремий автомобіль Audi A4 2020 року).

5. Що таке поле?

Поле (атрибут, колонка, column) — це іменована вертикальна складова таблиці, що характеризує окрему властивість об'єкта та має чітко визначений тип даних (наприклад, `price` типу REAL або `vin` типу VARCHAR(17)).

6. Що таке SQL?

SQL (Structured Query Language) — це декларативна мова програмування для взаємодії з реляційними системами керування базами даних (створення схем, читання, внесення змін, адміністрування транзакцій та керування доступом).

7. Для чого використовується CREATE TABLE?

Інструкція DDL `CREATE TABLE` використовується для створення нової таблиці в БД з оголошенням переліку колонок, їх типів даних та обмежень цілісності (PRIMARY KEY, NOT NULL, UNIQUE, FOREIGN KEY, CHECK).

8. Що означає CRUD?

CRUD — це акронім для чотирьох базових операцій із даними в інформаційних системах: **Create** (створення), **Read** (читання/вибірка), **Update** (оновлення), **Delete** (видалення).

9. Яка SQL-команда відповідає Create?

Команда DML `INSERT INTO`.

10. Яка команда відповідає Read?

Команда DQL `SELECT`.

11. Яка команда відповідає Update?

Команда DML `UPDATE ... SET ... WHERE ...`.

12. Яка команда відповідає Delete?

Команда DML `DELETE FROM ... WHERE ...`.

13. Що таке SQLite?

SQLite — це компактна, вбудована (in-process, serverless), кросплатформна реляційна СУБД із підтримкою транзакцій ACID, яка зберігає всю базу даних (схему, таблиці, індекси) в одному звичайному дисковому файлі або в пам'яті (`:memory:`).

14. Які переваги SQLite?

1. Відсутність окремого серверного процесу та конфігурації (Zero-Configuration).

2. Легкість розгортання та перенесення (один файл бази даних).
3. Висока швидкодія при локальних операціях читання/запису.
4. Вбудована підтримка у стандартній бібліотеці Python (`import sqlite3`).
5. Ідеально підходить для розробки, локального збереження налаштувань, мобільних застосунків та швидких ізольованих інтеграційних тестів у RAM.

15. Що таке Python DB-API?

Python DB-API (визначений у специфікації PEP 249) — це стандартний інтерфейс (API) мови Python для доступу до реляційних СУБД, що уніфікує взаємодію з різними рушіями баз даних (SQLite, PostgreSQL, MySQL) через стандартизовані об'єкти `Connection` та `Cursor`.

16. Для чого використовується Connection?

Об'єкт `Connection` представляє активний сеанс зв'язку між клієнтським застосунком і базою даних. Він керує життєвим циклом транзакцій (`commit()`, `rollback()`), створенням об'єктів курсорів (`cursor()`) та закриттям з'єднання (`close()`).

17. Для чого використовується Cursor?

Об'єкт `Cursor` призначений для безпосереднього виконання SQL-команд (`execute()`, `executemany()`), навігації та вичитування отриманих результатів із буфера (`fetchone()`, `fetchall()`, `fetchmany()`).

18. Для чого потрібен commit()?

Метод `commit()` зберігає (фіксує) всі зміни поточної транзакції у фізичній базі даних, роблячи їх постійними та видимими для інших користувачів та процесів.

19. Для чого потрібен rollback()?

Метод `rollback()` скасовує всі зміни, внесені з моменту початку поточної незавершеної транзакції, повертаючи базу даних у попередній узгоджений стан у разі збою чи помилки валідації.

20. Що таке SQL injection?

SQL Injection (SQL-ін'єкція) — це критична вразливість безпеки, що виникає при конструюванні SQL-запитів шляхом простої підстановки або конкатенації неперевірених користувацьких рядків у текст запиту, що дозволяє зловмиснику змінити логіку команди та отримати несанкціонований доступ до даних.

21. Що таке parameterized query?

Параметризований запит (Prepared Statement) — це спосіб оформлення SQL-запиту, при якому шаблон команди містить позиційні або іменовані плейсхолдери (`?` або `:param`), а самі значення параметрів передаються рушію СУБД окремим кортежем. Рушій компілює запит перед підстановкою, розглядаючи передані параметри виключно як літерали даних, що унеможливлює SQL-ін'єкції.

22. Чому не рекомендується формувати SQL через f-string?

Формування SQL через f-string створює пряму загрозу SQL-ін'єкцій, не виконує екранування спецсимволів (лапок, крапок з комою) та унеможливлює кешування плану виконання запитів СУБД.

23. Що таке Primary Key?

Primary Key (первинний ключ) — це стовпець або набір стовпців, значення якого однозначно ідентифікує кожний окремий рядок таблиці. Значення первинного ключа не може бути `NULL` та мусить бути строго унікальним.

24. Що таке Foreign Key?

Foreign Key (зовнішній ключ) — це стовпець або група стовпців однієї таблиці, які посилаються на первинний ключ іншої (або тієї ж самої) таблиці для встановлення реляційного зв'язку та забезпечення посилальної цілісності даних.

25. Що таке транзакція?

Транзакція — це логічна одиниця роботи з базою даних, що складається з однієї або кількох операцій (читання, запис, зміна), які повинні виконатися або повністю та неподільно, або не виконатися взагалі (атомарність).

26. Що означає Atomicity?

Atomicity (Атомарність) гарантує, що жодна транзакція не буде зафіксована частково. Якщо хоча б одна операція всередині транзакції зазнає збою, всі попередні зміни цієї транзакції відкочуються (`rollback`).

27. Що означає Consistency?

Consistency (Узгодженість) гарантує, що транзакція може перевести базу даних тільки з одного валідного стану в інший, не порушуючи встановлених бізнес-правил, типів даних, обмежень цілісності та зовнішніх ключів.

28. Що означає Isolation?

Isolation (Ізольованість) гарантує, що проміжні стани паралельно виконуваних транзакцій не впливають одна на одну і залишаються прихованими до моменту фіксації

(`commit`).

29. Що означає Durability?

Durability (Довговічність/Надійність) гарантує, що якщо транзакція успішно зафіксована (`commit`), її результати будуть збережені на дисковому носії та не втратяться навіть при раптовому вимкненні живлення чи аварії ОС.

30. Що означає ACID?

ACID — це фундаментальний акронім чотирьох властивостей транзакцій реляційних баз даних: **Atomicity** (Атомарність), **Consistency** (Узгодженість), **Isolation** (Ізольованість), **Durability** (Надійність).

31. Що таке persistence layer?

Persistence Layer (шар збереження даних) — це окремий архітектурний рівень застосунку, який відповідає за взаємодію зі сховищем даних (БД, файли), приховуючи технічні деталі роботи з СУБД від рівня бізнес-логіки.

32. Для чого потрібний Repository Pattern?

Repository Pattern інкапсулює логіку збереження, отримання та пошуку сутностей, імітуючи роботу з колекцією об'єктів у пам'яті. Це підвищує модульність, дозволяє легко замінювати реальне сховище тестовими макетами (Fake/Mock) та усуває дублювання SQL-коду.

33. Чому Repository відокремлюють від Service Layer?

Для дотримання принципу Single Responsibility Principle (SRP). Сервісний шар (Service Layer) містить бізнес-правила, розрахунки та оркестрацію процесів, тоді як Repository відповідає лише за читання та запис даних. Їх розмежування спрощує модульне тестування бізнес-логіки без підключення до реальної бази.

34. Що таке ORM?

ORM (Object-Relational Mapping) — це технологія програмування, яка зіставляє реляційні таблиці бази даних із класами мови програмування, а рядки таблиць — з об'єктами цих класів, дозволяючи розробнику працювати з базою засобами об'єктно-орієнтованого коду.

35. Для чого використовується SQLAlchemy?

SQLAlchemy — це найпопулярніша бібліотека мови Python для роботи з реляційними базами даних. Вона поєднує низькорівневий рушій взаємодії Core (SQL Expression Language, пул з'єднань, діалекти) та високорівневу систему об'єктно-реляційного відображення ORM.

36. Що таке ORM model?

ORM Model — це клас у Python, що успадковує `DeclarativeBase` та описує схему таблиці: назву таблиці (`__tablename__`), перелік стовпців, їх типи даних, первинні та зовнішні ключі, а також зв'язки з іншими моделями.

37. Що таке relationship()?

Функція `relationship()` у SQLAlchemy оголошує логічний зв'язок між двома ORM-моделями (one-to-one, one-to-many, many-to-many), дозволяючи автоматично завантажувати пов'язані об'єкти як атрибути-колекції та керувати каскадним видаленням.

38. Що означає one-to-many?

Зв'язок «один-до-багатьох» (1:N) означає, що один запис у батьківській таблиці може бути асоційований із багатьма записами в дочірній таблиці (наприклад, один виробник `Manufacturer` асоційований із багатьма авто `CarEntity`), тоді як кожне авто належить лише одному виробнику.

39. Що означає many-to-many?

Зв'язок «багато-до-багатьох» (M:N) означає, що один запис першої таблиці може посилатися на декілька записів другої таблиці, і навпаки. Реалізується через додаткову проміжну асоціативну таблицю (junction table) із двома зовнішніми ключами.

40. Що таке migration?

Міграція — це версійний файл-сценарій, що програмно описує зміни в структурі схеми бази даних (додавання або видалення таблиць, колонок, індексів) із можливістю застосування змін (`upgrade`) або повернення до попередньої версії (`downgrade`).

41. Для чого використовується Alembic?

Alembic — це спеціалізований інструмент керування міграціями для SQLAlchemy, який дозволяє відстежувати зміни в ORM-моделях, автоматично генерувати ревізії міграцій (`autogenerate`) та послідовно синхронізувати схему фізичної БД з кодом.

42. Для чого потрібна команда alembic upgrade head?

Команда `alembic upgrade head` застосовує всі наявні, ще не виконані міграції з каталогу `migrations/versions/` до цільової бази даних, оновлюючи її схему до найновішого стану (`head`).

43. Що таке integration test?

Інтеграційний тест (Integration Test) — це тест, що перевіряє спільну коректну роботу кількох взаємопов'язаних компонентів системи (наприклад, Repository + SQLAlchemy ORM + рушій SQLite бази даних).

44. Чим integration test відрізняється від unit test?

Unit-тест ізольовано перевіряє один окремий метод або функцію, замінюючи всі зовнішні залежності макетами (Mock/Fake). Інтеграційний тест перевіряє реальну взаємодію компонентів із зовнішніми підсистемами (реальна база даних, файлова система, мережа).

45. Навіщо використовувати окрему test database?

Використання окремої тестової БД (наприклад, `sqlite:///:memory:`) гарантує повну ізоляцію від робочих (production) даних, швидке виконання тестів без залишкового сміття, відсутність конфліктів між запусками та безпеку даних розробника.

46. Що перевіряє тест rollback?

Тест перевіряє, що у разі виникнення винятку під час виконання транзакційної операції всі проміжні зміни дійсно скасовуються, і база даних не містить частково оновлених чи некоректних даних (перевірка принципу Atomicity).

47. Для чого використовуються fixtures?

Pytest fixtures використовуються для стандартизованої підготовки тестового середовища (створення бази даних, ініціалізація схеми, заповнення демо-даними) та гарантованого звільнення ресурсів (`yield` + teardown/cleanup) після завершення тесту.

48. Що робить SQL JOIN?

Оператор `JOIN` поєднує рядки з двох або більше таблиць на основі зв'язаного між ними стовпця (зазвичай зовнішнього і первинного ключів), формуючи єдину комбіновану вибірку результатів.

49. Що робить GROUP BY?

Речення `GROUP BY` групує рядки вибірки, що мають однакові значення у зазначених стовпцях, в агреговані підсумкові записи, дозволяючи застосовувати до кожної групи агрегатні функції.

50. Для чого використовуються COUNT, SUM, AVG, MIN, MAX?

Це агрегатні функції мови SQL: - `COUNT()`: підраховує кількість рядків у вибірці або групі; - `SUM()`: обчислює загальну суму числових значень; - `AVG()`: розраховує середнє арифметичне значення; - `MIN()`: знаходить мінімальне значення у стовпці; - `MAX()`: знаходить максимальне значення у стовпці.

---

28. Висновок

У процесі виконання лабораторної роботи № 7 було успішно реалізовано повноцінний шар збереження даних (Persistence Layer) на базі вбудованої СУБД SQLite та сучасної ORM-бібліотеки SQLAlchemy 2.0.

Основні результати роботи: 1. Спроєктовано реляційну схему для каталогу автомобілів (Варіант 5), що містить сутності `manufacturers` та `cars`, зв'язані відношенням «один-до-багатьох» із підтримкою первинних ключів, обмежень унікальності (`UNIQUE`), валідації діапазонів (`CHECK`) та каскадного видалення (`ON DELETE CASCADE`). 2. Налаштовано примусове дотримання зовнішніх ключів у SQLite через подію з'єднання `PRAGMA foreign_keys = ON;`. 3. Імплементовано патерн Repository (`SqlAlchemyManufacturerRepository`, `SqlAlchemyCarRepository`), що забезпечив повне відокремлення доменної логіки від деталей взаємодії з СУБД. 4. Продемонстровано застосування стандарту Python DB-API (PEP 249) з обов'язковою параметризацією запитів для захисту від SQL-ін'єкцій. 5. Розроблено транзакційну операцію масового перерахунку цін `bulk_update_prices_by_manufacturer` із явним контролем атомарності через `commit()` та `rollback()` згідно з принципами ACID. 6. Впроваджено систему міграцій схеми даних Alembic із двома послідовними ревізіями та протестовано пряме й зворотне оновлення схеми. 7. Розроблено набір із 12 інтеграційних тестів із тестовою in-memory базою даних. Усі 146 тестів проєкту проходять успішно із покриттям коду 85%, а статичний аналізатор `mypy` підтверджує строгу типізацію на 50 файлах коду без жодної помилки.